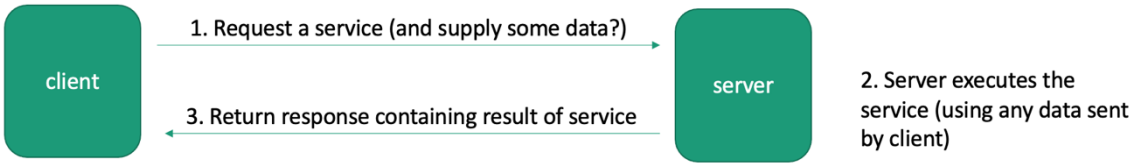
6COSC022W Advanced Server-Side Web Programming
Lecture 1 – Web Applications

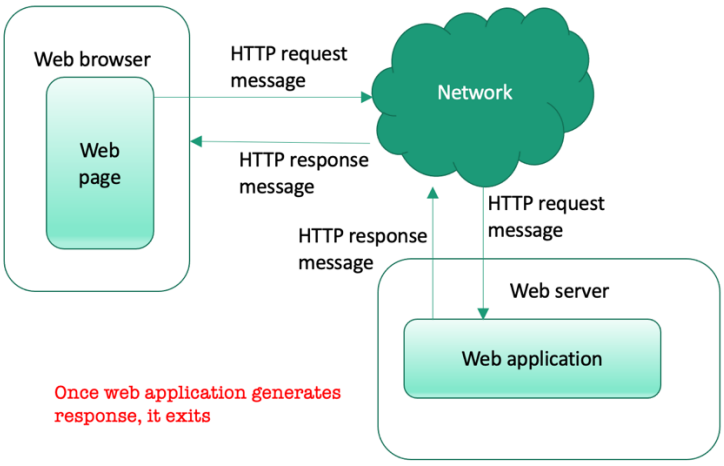
What are web applications?

Web applications are *client-server* applications. Client-server applications are distributed applications because they are made of different parts which are typically distributed across different machines connected to the same network. Client-server refers to the particular roles taken on by different parts of the application: one part acts as a "client", another acts as the "server", with clients making requests or asking for something to be done, and the server fulfilling those requests and responding to them.



This diagram shows a single application. But it has been split into two parts, one part playing the role of a client, who makes requests, and the other part playing the role of server, who fulfills those requests and sends back some response. The client-server application model is perhaps the most common kind of distributed application – some other examples of client-server applications are FTP programs (such as FileZilla and CyberDuck) and mobile applications for messaging (such as WhatsApp) or banking.

In terms of web applications, the client part is that part of the application which is in the user's browser. The server part of the application is the server-side code. Typically, the browser-side part of the application runs on a user's own machine, while the server-side part of the application runs under a web server on a different machine, which is reachable via the network. (Note that I will sometimes call the part of the web application which runs in the browser as the "client-side code" or just "the client").



In client-server applications, the different parts work together by sending each other messages across the network. This message-passing communications requires:

1. That the client knows the identity or location of the server

2. The client (web browser) and (web) server know how to understand each other's messages

In web applications, the first requirement is met by URLs, while the second requirement is met by HTTP, the Hypertext Transfer Protocol.

URLs

URLs are the means by which the client-side code of your web application knows where to find the server-side part of the application. It is also how the browser first locates your web application to load in the client-side code. URLs are *addresses* of things on the web called *resources* – hence URL stands for Uniform Resource Locator. The kind of things that a URL addresses include HTML files, PHP programs, CSS and Javascript files, images and so on. Each of these things, if they are to be found on the web, needs their own URL.

A URL has many parts, just as any kind of address does. Here's an example:

http://www.example.com/shop/products/browse.php?category=laptops&maxprice=500

This URL is made up of:
A protocol identifier – in this case it is "http", indicating that the messages to be used to fetch the resource with this address will be formatted using HTTP. Other protocols might be "https" (secure HTTP) or "ftp" (if we wanted to communicate with an FTP server for this resource)

"://" – this is just a separator between the protocol and the resource address itself

The domain name of the web server managing the resource – here it is www.example.com. Domain names follow a hierarchial naming format – ".com" is the "top-level" domain (TLD) component (https://en.wikipedia.org/wiki/.com). The domain name actually identifies a machine on the internet via a domain name-to-IP address mapping. That machine will (we hope) be running a web server that will react to our HTTP requests.

When a networked machine receives a message from the network, it needs to know, out of all the programs it is running, which program should be given that message. This is decided using port numbers. Each running program that connects to the network has a *port number* allocated to it – very common applications such as web servers typically have the same port number on every machine. Web servers typically use port 80. We could type the URL above as "http://www.example.com:80/", but because it is so common to use port 80 for web servers, browsers just assume port 80 if you don't type it. Hence "http://www.example.com/" is just shorthand for "http://www.example.com:80/". If your web server is running on any other port, then the port number must be made part of the URL. This is most common when you are running your web server on your own machine using localhost (perhaps via XAMPP or MAMP) – then you might need to use "http://localhost:8080".

The path – "/shop/products/browse.php" – which identifies where on the machine running the web server we can find the resource (in this case, a PHP file). This path is always relative to the web server document root folder. So if the web server document root folder is "/home/web/htdocs" on the machine's file system, then the full filesystem path to the file will be "/home/web/htdocs/shop/products/browse.php".

The query string – query strings are not always part of a URL, but if they are, they always come at the end, separated from the rest of the URL by a '?' character. They are one of the ways we can pass data to a server-side program. See https://en.wikipedia.org/wiki/Query_string for more details.

A URL like the one above is called an *absolute* URL, because it is complete – it contains all the information needed to provide the address of a resource on the web. But a URL can also be relative – in which case, it only contains the path element (and possibly query string data). For example, suppose we have a web page at http://www.example.com/store/basket/index.html which contains the following HTML code:

```
<img src="/images/one.gif" />
```

The value of the src attribute is a URL, but it is not an absolute URL. It only has the path to the resource from the web server document root URL. Note that it begins with "/" – which tells us to start the path from the document root folder. We can work out the absolute URL, however, by adding the domain of the URL of the containing page – hence the URL is really http://example.com/images/one.gif.

Another type of relative URL is this:

```
<img src="../images/one.gif" />
```

If this tag is embedded in the page at http://www.example.com/store/index.html, then the ".." at the start tells us to go one folder up from the current folder (the one containing index.html) – so the full URL of the gif file is http://www.example.com/store/images/one.gif.

HTTP (Hypertext Transfer Protocol)

The second requirement is that the browser and the web server understand each other's messages. For a program to understand a message means that it knows the structure of the message and is able to process it using rules built into its code.

The Hypertext Transfer Protocol defines what messages can be sent between a browser and a web server and how they are structured. In essence, there are two types of messages defined in HTTP – the request message sent by the browser to the server, and the response message sent by the web server to the browser.

We don't usually see the messages sent by the browser and the server to each other, but you can view them using your browser's developer tools. Each request and response message is split into two parts: the headers and the payload (separated by a blank line). The request contains headers such as:

```
GET https://www.theguardian.com/uk
user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/105.0.0.0 Safari/537.36
```

It might also have a header telling the server what page the request is coming from (i.e., what is the current page in the browser). The response headers look like this:

```
Request URL: https://www.theguardian.com/uk
Request Method: GET
Status Code: 200
content-length: 168725
content-type: text/html; charset=UTF-8
date: Wed, 21 Sep 2022 14:47:07 GMT
```

(plus the HTML code after a blank line in the message).

The status code 200 tells the browser that the request was fulfilled successfully. Other status codes tell the browser, for example, that the URL is being redirected, that the request could not be fulfilled, or that there was an error – see https://developer.mozilla.org/en-US/docs/Web/HTTP/Status for more.

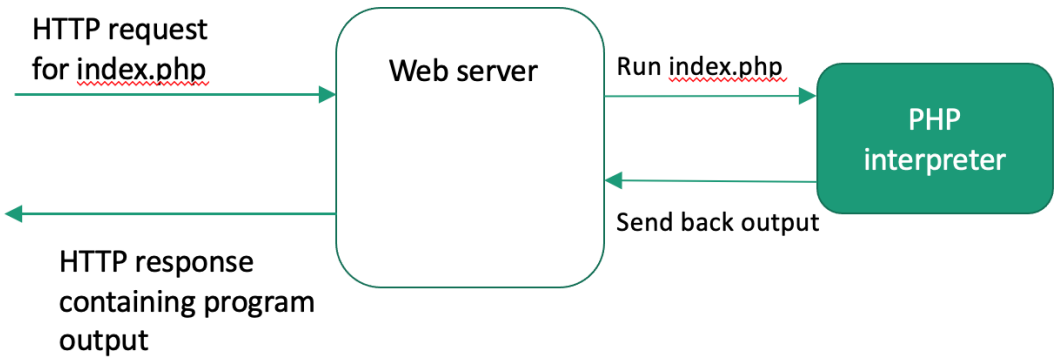
The most important part of HTTP for our purposes is the request method – for example, the GET in the request message example above. Request methods tell the server what kind of action to carry out – typically web applications use GET and POST, but there are others: https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods. We will see the use of PUT and DELETE when we come to discuss REST APIs.

When a web server receives an HTTP request, it performs certain actions to meet the request. Typical actions involve

- Locating the resource named in the request message
- Deciding whether to return the content of the resource (for example, if it is a static file such as an image) in the response, or to execute the resource as a program and return its output in the response
- Sending back the response

The web server distinguishes between static files whose content is sent back (such as HTML and image files) and dynamic content, which are executed as programs on the server and whose output is returned. The server uses the file extension of the resource to make this distinction. Unless told otherwise, the server treats a resource as a static file, but if a file extension (such as ".php") is registered with the server (using its config file) as a dynamic resource, it will then execute it using the information provided in its config file.

Whatever action is carried out, if it is successful, the web server sends back a response with a 200 status code, a content-type header to tell the browser what kind of data to expect and the data itself.



Writing Web Applications

There are two main concerns we need to bear in mind when writing web applications, concerns we do not typically deal with when working with other types of application. The first is how the application is structured. The second is what tools, languages and technologies we use.

Application Structure
Web applications are a mix of static and dynamic (program-generated) content. What we call a "web application" may be made up of static pages, CSS files, javascript files, images, as well as many different program files.

A typical example of a web application is an ecommerce website. The application allows people to find products, see reviews, put products into a shopping basket, and checkout and pay for the items in their basket. But the application is not one monolithic program, but several programs and static files, each doing a particular part of the application. In these types of application, the different programs need to work together – the browsing part of the ecommerce application needs to show products that the basket program can put into the basket, and so on. The way these programs work together is by sharing a database – each program uses the same database.

If we were to write a desktop application or a mobile application, it would be one program with everything needed included in the code of the program. But web applications are split into many different files, typically stored in different folders under the web server document root folder. This makes web applications more complex to build than traditional programs.

Tools, technologies etc.

The other concern we have as web developers is with the profusion of tools, technologies and frameworks we have to deal with. Web applications typically involve at least two different platforms: the browser and the machine running the web server. The browser, as a platform, understands only HTML, CSS and Javascript as technologies, while the web server's platform is much more flexible in terms of programming language: we can use PHP, Python, Java, Ruby – in fact, we can more or less use any programming language we want. Then there is the database – MySQL, PostgresSQL, MongoDB are among the options we can select as a data store.

Being what is now called a "full stack" web developer means, then, being able to use a much wider range of languages than other types of developer. And on top of that, we have frameworks – each programming language has its own family of frameworks. PHP has CodeIgniter, Zend, Symfony, Laravel, Yii, CakePHP, among others, Python has Django and Flask, Javascript has React, Backbone and many more.

On this module, we will be using PHP, mostly with CodeIgniter, and Javascript, with the Backbone framework. PHP is an interesting language – apart from Javascript, it is probably the only language created specifically for web development.

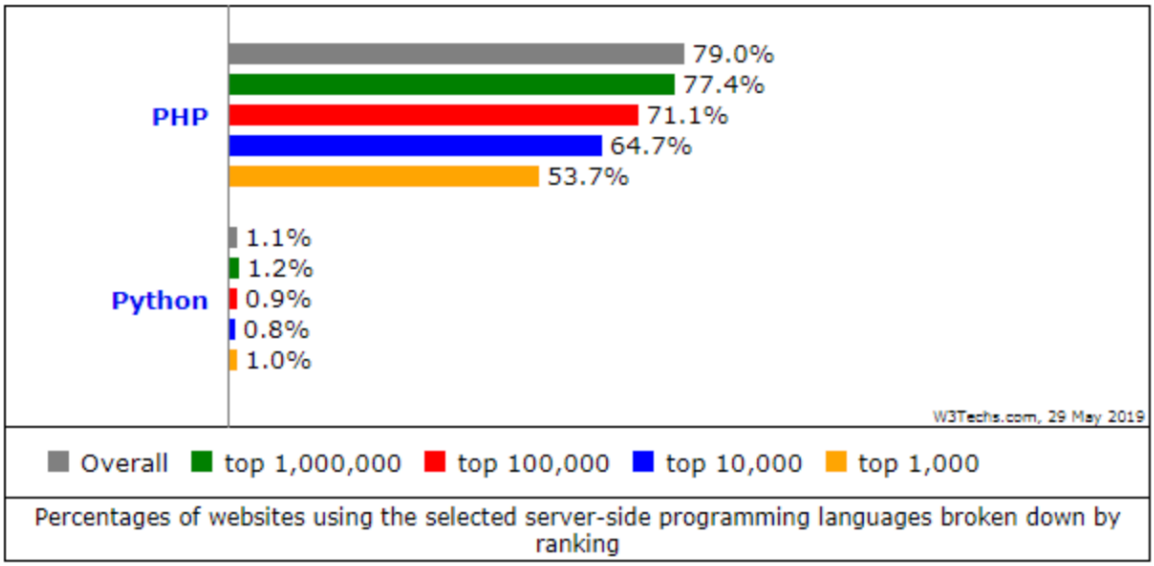
The first version of PHP, called PHP/F1 (or PHP1), was created by Rasmus Lerdorf in 1994 and looked like this:

```
<!--include /text/header.html-->

<!--getenv HTTP_USER_AGENT-->
<!--if substr $exec_result Mozilla-->
  Hey, you are using Netscape!<p>
<!--endif-->

<!--sql database select * from table where user='$username'-->
<!--ifless $numentries 1-->
  Sorry, that record does not exist<p>
<!--endif exit-->
  Welcome <!--$user-->!<p>
  You have <!--$index:0--> credits left in your account.<p>

<!--include /text/footer.html-->
```

(from Wikipedia: https://en.wikipedia.org/wiki/PHP#Early_history)

We are now on PHP7 – a version that includes support for object-oriented programming and namespaces. PHP7 also greatly improves performance over PHP5, through an improved interpreter engine – this is significant because performance, measured in terms of CPU time, has always been one of PHP's weaknesses.

Regarding usage in web development, PHP remains one of the most commonly-used languages – as this chart from https://www.clariontech.com/blog/python-vs.-php-the-battle-is-on shows, more websites use PHP than Python in their stack.



(chart from ClarionTech at https://www.clariontech.com/blog/python-vs.-php-the-battle-is-on)